

Article

Android Malware Detection Based on Behavioral-Level Features with Graph Convolutional Networks

Qingling Xu ^{1,2}, Dawei Zhao ^{1,2} , Shumian Yang ^{1,2,*}, Lijuan Xu ^{1,2} and Xin Li ^{1,2}

¹ Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China; 10431210700@stu.qlu.edu.cn (Q.X.); zhaodw@sdas.org (D.Z.); xulj@sdas.org (L.X.); lixin@sdas.org (X.L.)

² Shandong Provincial Key Laboratory of Computer Networks, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China

* Correspondence: yangshm@sdas.org

Abstract: Android malware detection is a critical research field due to the increasing prevalence of mobile devices and apps. Improved methods are necessary to address Android apps' complexity and malware's elusive nature. We propose an approach for Android malware detection based on Graph Convolutional Networks (GCNs). Our method focuses on learning the behavioral-level features of Android applications using the call graph extracted from the application's Dex file. Combining the call graph with sensitive permissions and opcodes creates a new subgraph representing the application's runtime behavior. Subsequently, we propose an enhanced detection model utilizing graph convolutional networks (GCNs) for Android malware detection. The experimental results demonstrate our proposed method's high precision and accuracy in detecting malicious code. With a precision of 98.89% and an F1-score of 98.22%, our approach effectively identifies and classifies Android malicious code.

Keywords: graph convolutional networks; Android; malware detection; call graph



Citation: Xu, Q.; Zhao, D.; Yang, S.; Xu, L.; Li, X. Android Malware Detection Based on Behavioral-Level Features with Graph Convolutional Networks. *Electronics* **2023**, *12*, 4817. <https://doi.org/10.3390/electronics12234817>

Academic Editor: Andrei Kelarev

Received: 24 October 2023

Revised: 21 November 2023

Accepted: 23 November 2023

Published: 28 November 2023

Corrected: 14 November 2024



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Mobile devices have become an integral part of our daily lives, with smartphones capturing a substantial market share that has grown from 1% to 50% in the past decade [1]. Android holds the largest market share, making it the most popular choice for users. Unfortunately, this popularity has also attracted the attention of malicious actors, resulting in a staggering number of Android malware programs. Android malware often disguises itself as innocuous applications or directly targets the device's operating system, posing significant security risks and potential data breaches [2]. Consequently, combating Android malware has emerged as a prominent challenge in mobile security.

Existing analysis techniques for detecting Android malware can be broadly categorized into two main approaches: dynamic [3,4] and static [5,6]. Dynamic techniques provide valuable insights into the runtime behavior of applications, offering valuable clues for malware detection. However, they have the drawback of requiring the monitoring of application execution, which can introduce overhead and inconvenience. On the other hand, static features can be obtained only by analyzing the application installation file, known as the Android application package (APK). This approach saves time and reduces costs by avoiding the need for runtime monitoring. In previous literature [7–9], researchers commonly employed static features such as permission requirements, intent operations, and function calls (API calls) for malware detection, and although these static features provide valuable information, they may need to capture the runtime behavior of applications accurately. We propose a novel approach that focuses on learning behavior-level features extracted from call graphs to address the limitation. As malware increases in sophistication,

traditional feature engineering and machine learning methods face difficulty detecting novel attacks. This has motivated a transition to graph-based and deep learning techniques like GNNs that can capture more complex feature representations and relationships.

Function calls in Android applications are commonly represented as binary vectors or directed graphs, known as API call graphs [10]. In an API call graph, each node represents an API call, whereas the edges depict the call relationships between APIs. This graphical representation offers valuable insights into the application's functionality and significantly contributes to effective malware detection. However, using call graphs in Android malware detection encounters two significant challenges. Firstly, relying solely on the call graph representation extracted from the APK file may not capture the complete behavioral semantics of the application, and although call graphs provide valuable information about the interactions between APIs, they may not fully represent the intricacies of the application's runtime behavior. Secondly, classical machine learning algorithms are not well-equipped to handle graph structures due to their non-Euclidean nature [11]. This presents a significant hurdle in effectively leveraging call graphs for Android malware detection.

In response to the challenges outlined earlier, MsDroid [12] extracts call graphs and semantic features such as permissions and opcodes to augment the understanding of application behavior. It employs a Graph Neural Network (GNN) model to process the graph structure features directly, enhancing the accuracy and universality of Android malware classification. However, the MsDroid simplified call graph method is relatively complex. In light of this, we introduce a novel approach for simplifying call graphs, aiming for more accurate localization and capture of code structures closely associated with sensitive permission behaviors. Additionally, relying solely on a two-layer GNN for feature representation, as employed by MsDroid, is considered inadequate due to the need for a more comprehensive consideration of inter-node relationships. Addressing these concerns, we propose a new GCN model that better captures the inter-node relationships and essential features. This improvement enhances the efficiency of feature propagation, making the new model more comprehensive and accurate in handling graph data. These enhancements result in superior performance in malicious software detection. Our research makes the following contributions:

- Our proposed Android malware detection method uses the Androguard [13] tool to extract call graphs from APK files. By combining features such as permissions and opcodes, our approach captures the call relationships between APIs and incorporates the semantic information of APIs, while simplifying the call graph construction method. This comprehensive representation enables us to adaptively explore and emphasize potential feature combinations or dependencies from multiple perspectives, improving malware detection performance.
- To address the Android malware detection challenge, we employ an enhanced graph convolutional network (GCN) framework for Android malware detection. Our framework directly utilizes the call graph as the input for the GCN, eliminating the need to compress the graph data into low-dimensional vectors. By doing so, we preserve the complete graph structure information, transforming the malware detection problem into a graph classification problem. This approach enhances the accuracy and reliability of our detection system.
- Extensive experiments were conducted to evaluate the effectiveness of our method. We explored different GCN algorithms, GCN layers, and graph semantic features. Compared to existing methods that rely solely on call graph features, our proposed approach demonstrated higher detection precision and a lower false positive rate. The results of our extensive experiments indicate that our method outperforms state-of-the-art techniques in Android malware detection.

Based on the comprehensive analysis, this research concludes that the feature fusion method indeed contributes to an enhancement in the accuracy of classifiers for classification. The model also introduces a fusion based on a graph convolutional model, enabling the capture of more feature information. This method offers a new approach to the fusion

of Android malicious software features, supplementing existing methods and providing a fresh perspective for further improvements in deep learning-based Android malware detection systems. It holds the potential to become a powerful tool for security vendors and antivirus platforms, strengthening the overall security of the mobile ecosystem and safeguarding the information of numerous mobile users.

2. Related Work

We have conducted research focusing on the latest developments in the following areas: malware detection and classification, graph feature extraction, and graph neural networks.

2.1. Malware Detection Based on Traditional Features

Numerous studies have focused on static analysis of malware, employing different types of feature sets as inputs to machine learning (ML) [14,15] and deep learning (DL) [16,17] models. The performance of malware detection heavily relies on the selected features, and these methods utilize classifiers to determine whether an application is malicious. Initially, low-level primitive features were employed in malware detection, such as API call sequences extracted by Aafer et al. [18], and analysis of source code API-level features like package names and class names for malware detection. Deshotels et al. [19] proposed a method that used API calls as feature vectors for machine learning classifiers. Kim et al. [20] combined extracted strings with deep-learning techniques for Android malware detection. Additionally, other features like intent [21] and permissions [22] have also been incorporated. However, these individual features are vulnerable to obfuscation techniques, which can adversely affect the detection performance. As a result, graph-based feature detection methods have emerged as a promising approach.

However, these traditional features, such as API call sequences and source code API-level features, were initially widely applied. Yet, with the increasing complexity of malicious software and the continuous evolution of obfuscation techniques, these features have encountered challenges in capturing novel malware. Obfuscation techniques can undermine the effectiveness of these features, leading traditional methods to perform poorly in detecting and classifying the latest malicious software. Graph features, on the other hand, exhibit certain advantages when dealing with common obfuscation techniques in malware. Due to the representation of malicious software behavior and structure in a graph, this approach is more flexible and less susceptible to simple obfuscation techniques. Therefore, in the following sections, we explore methods that utilize graph features to detect malicious software, aiming to capture these intricate patterns of malicious behavior more accurately.

2.2. Graph-Based Malware Detection

Graph-based approaches in malware detection have gained attention for robust and rich feature representations. Examples include M. Fan et al. [23], who modeled behavior as subgraphs for detecting malicious activity. Fan et al. [24] extracted call and data flow graphs to identify malicious subgraph features. Narayanan et al. [25,26] used control flow graphs to classify malware, whereas Hassen et al. [27] employed function call graphs for effective detection. These techniques capture comprehensive information for accurate malware detection.

Graph-based features have shown powerful capability in representing characteristics for malware detection. However, they also have some limitations. A single graph structure may not provide sufficient contextual or relational information to cover the diverse behavioral patterns of malware. To compensate for these deficiencies, we combine other semantic features based on original graph features, which helps expand the breadth and depth of graph features, further enriching the characteristic expression of malware. This enables us to more comprehensively capture subtle features of malicious behaviors while improving the accuracy of malware detection models.

2.3. Machine Learning (ML) and Deep Learning (DL) Models

Various ML-based methods have been developed for Android malware detection, including K-NN, SVM, NB, and RF. For instance, Chen T, Mao Q, et al. [28] proposed TinyDroid, which utilizes static analysis and ML techniques with RF, SVM, K-NN, and NB for classification. Deep learning models like DL-Droid by Alzaylaee MK et al. [29] use CNNs and RNNs to detect and classify Android malware. Hemalatha J et al. [30] introduced a DenseNet-based model for Android malware detection, converting binary code to images and performing feature extraction and classification. These examples highlight the use of ML and DL models for accurate Android malware detection.

The comparison of methods proposed by different researchers and their limitations is shown in Table 1. Compared to solely extracting individual static and dynamic features, utilizing function call graphs with node features and semantic information allows for inferring behavioral characteristics of applications using neighborhood details and semantics, whereas code obfuscation affects static feature analysis, function renaming does not alter the topological structure of function call graphs. This method mitigates the impact of function renaming and circumvents issues associated with high costs in dynamic feature experiments and the difficulty in triggering all malicious behaviors within an application.

Table 1. Comparison with related work.

Author	Year	Feature	Algorithm Model	Limitation
Narayanan et al. [25]	2016	Code Graphs	SVM Algorithm	Relies on the manual design of feature representation and may not generalize well to new malware types or attacks.
Xu et al. [26]	2017	Control Flow Graph	Neural Network-Based Graph Embedding	Require substantial computing resources
Hassen et al. [27]	2017	Function Call Graph	Random Forest Algorithm	Only considers the context of function call relationships without considering more semantic information in the program.
Chen et al. [28]	2018	N-gram sequences	Various classic machine learning algorithms	Only utilizing code-level syntactic features, without capturing program semantics information.
Alzaylaee et al. [29]	2020	Application attributes, Actions-/Events, Per-mission	Deep learning algorithms	Using dynamic features for malicious code detection include an inability to cover all potential behaviors, sensitivity to time, and susceptibility to adversarial attacks
Hemalatha et al. [30]	2021	Binary images	DenseNet model algorithms	Unable to comprehensively capture the code semantics

With the widespread application of deep learning in malware detection, the integrated learning framework based on deep neural networks has also become a hotspot direction. Azeez et al. [31] proposed a two-stage combined learning method for detecting Windows PE malware. This method used fully connected neural networks and one-dimensional convolutional neural networks as base learners in the first stage. In contrast, machine learning classifiers were used as meta-learners in the second stage. By comparing 15 different meta-learners, it was found that the ExtraTrees classifier could achieve optimal detection performance. Damaševičius et al. [32] also proposed an integrated learning method for Windows PE malware detection based on ensemble learning, using neural networks as base learners and machine learning algorithms as meta-learners, proving this framework's effectiveness in improving detection performance. Compared with single models, ensemble learning better uses the advantages of different models and enhances the robustness of the detection system. This provides valuable inspiration for our work, and we also consider exploring the framework of ensemble learning based on GCN models, expecting similar performance improvements.

2.4. Graph Neural Networks

Graph neural networks have been extensively utilized in developing classifiers for Android malware detection. These networks extract informative features from APK files and represent them as graph structures, enabling the modeling of complex characteristics of malware and creating more accurate classification models during training.

For example, Zhang et al. [33] proposed a graph convolutional network for semi-supervised classification of graph-structured data, achieving good performance in protein family classification. Gao and Cheng [34] introduced GDroid, a graph convolutional network-based method for Android malware detection, which effectively converted APK files into API call graphs and extracted local, global, and sequence features for classification. Yan et al. [35] proposed a deep graph convolutional neural network approach that utilized attention mechanisms to classify malicious software represented by control flow graphs.

Graph neural network models can effectively process graph-structured features and capture relationships and contextual information between nodes. Therefore, in this paper, we extend the GCN model to a more comprehensive framework by introducing LSTM layers and attention mechanisms into the GCN model. This enables the processing of graph data and a better understanding of sequential and global correlations. Such integration can more accurately capture long-term dependencies between nodes and allow the model to focus on critical connections, thereby improving the model's understanding of node relationships and contextual information. With this improvement, our method has more advantages in dealing with function call graph data and further enhances the efficacy and performance of our process. This enhancement of GCNs enables our approach to make significant progress in addressing some limitations of function call graphs and GCN models, which are validated by satisfactory experimental results.

3. Methodology

The overall framework of our proposed Android malware detection method is shown in Figure 1. It consists of three stages. The first stage is preprocessing the dataset and feature engineering. The second stage is generating behavior-level features. In this stage, similar to MsDroid [12], we extract API call graphs, opcodes, and sensitive to enhance API semantics. Furthermore, we propose a novel GCN model to explore the relationships among these features in the constructed subgraph structure based on the form of the call graph. In the third stage, we employ the enhanced GCN model for classification and detection.

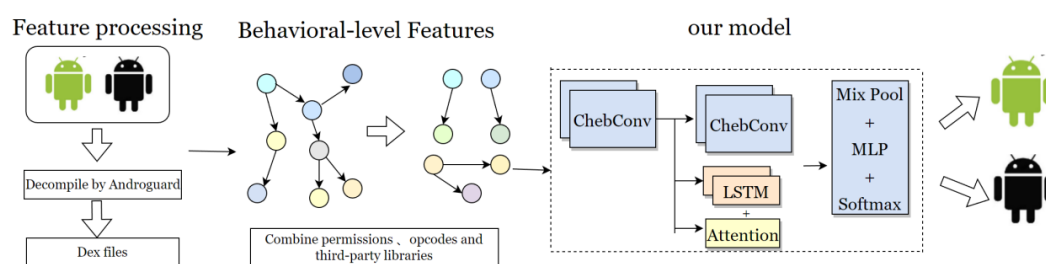


Figure 1. The overall framework.

3.1. Feature Extraction

We focus on extracting relevant features from the Android APK package in the feature extraction stage. The Manifest file, which contains essential configuration information for the application, is analyzed to extract necessary components, permissions, and other related details. Additionally, the Dex files within the APK, which are Dalvik executable files, provide valuable semantic features of API calls.

To safeguard the accuracy of the feature extraction process, we proactively favor this approach over extracting features from the Manifest.xml file, considering that permissions removed from the manifest can be readily obfuscated. Leveraging Androguard can obtain more reliable and meaningful features for further analysis and classification.

3.1.1. API Call Graph

The call graph, denoted as $G = (V, E)$, is integral to our methodology. In this context, each element $v \in V$ represents an API, and each edge $e = (v_1, v_2) \in E$ represents a call from v_1 to v_2 . We utilize the Androguard tool's AndroguardGraph class from the Androguard.misc module to generate the call graph. By processing the Dex file, the AndroguardGraph class generates a Graph Modeling Language (GML) file. This GML file serves as a representation of the API call graph within the Android application. Nodes in the graph correspond to classes or methods, whereas the edges capture API call relationships.

Figure 2 is an Android code call graph related to command execution and ROOT permissions.

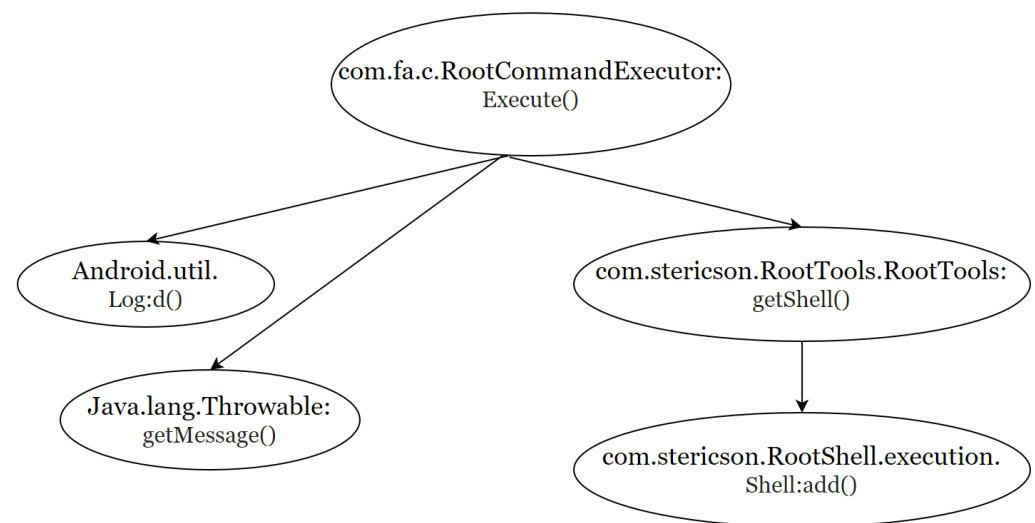


Figure 2. An example of a simple API call graph.

This call graph has three classes: RootCommandExecutor, RootTools, and RootShell. The graph shows that the Execute() method of RootCommandExecutor calls the getShell() method of RootTools. In a broader perspective, this call graph illustrates an execution flow initiated by RootCommandExecutor, progressively invoking RootTools and RootShell. We utilize this flow for the execution of operations that necessitate ROOT permissions and to capture exceptional information during the execution process.

The resulting API call graph enables us to analyze the relationships between different methods within the application. By analyzing these nodes and edges, one can understand the API call relationships in the application, which can help understand the structure and functionality of the application.

3.1.2. Permissions

Permissions are widely employed in Android malware detection to restrict app access to sensitive user data, such as contacts and SMS, as well as specific system actions like camera and Bluetooth access. API calls in Android applications are directly associated with permissions, as sensitive APIs typically necessitate corresponding permissions. As a result, developers can utilize the combination of API calls and permissions to characterize Android applications with valuable semantics.

To extract permission information from the API call graph of an Android application, we parse node information and reference existing permission mappings. In this study, two widely used API-permission mapping tools, PSCout and Axplorer, were employed. These tools conduct static analysis on the application's code to establish associations between APIs and their respective permissions. Table 2 depicts partial API-to-Permission mappings.

Moreover, incorporating specific sensitive permissions into the graph representation significantly fortifies the capability to discern between benign and malicious behaviors

exhibited by Android applications. For instance, permissions such as READ_SMS and READ_CONTACTS directly relate to actions that often indicate potential risks when present in an application's behavior. Integrating these permissions into the graph structure becomes pivotal, as they serve as distinct markers that shape the app's behavioral landscape within the representation.

Table 2. API-permissions mapping.

Number	Permissions	Mapped APIs
1	Landroid/content/ContentService;->getMasterSyncAutomatically(L;)Z	android.permission.READ_SYNC_SETTINGS
2	Landroid/content/ContentService;->getIsSyncable(Landroid/accounts/Account;Ljava/lang/String;)I	android.permission.READ_SYNC_SETTINGS
3	Landroid/content/ContentService;->getCurrentSyncs(L;)Ljava/util/List;	android.permission.READ_SYNC_STATS
4	Landroid/media/AudioService;->setSpeakerphoneOn(Z)V	android.permission.MODIFY_AUDIO_SETTINGS
5	Landroid/media/AudioService;->setBluetoothScoOn(Z)V	android.permission.MODIFY_AUDIO_SETTINGS
6	Landroid/server/BluetoothService;->cancelDiscovery(L;)Z	android.permission.BLUETOOTH_ADMIN

By embedding these sensitive permissions within the graph nodes or edges, the representation becomes more nuanced and capable of discerning nuanced behavioral patterns. Consider an application requesting access to sensitive data like reading SMS without an apparent need based on its primary function; this incorporation of permission into the graph representation aids in highlighting discrepancies or potential malicious intentions. Therefore, this in-depth inclusion of sensitive permissions is fundamental in enriching the graph representation, enabling a more robust analysis of an application's behavior for potential threat detection.

3.1.3. Opcode

Opcodes play an essential role in representing compiled Android apps. As attributes of sensitive subgraph nodes, they provide vital information to extract operations and behaviors within applications. Therefore, selecting and utilizing these opcodes as attributes of sensitive subgraph nodes is crucial to giving detailed insights into functions within applications.

Opcodes serve as representations of compiled Android applications using mnemonic instructions. In this study, we employ opcodes as attributes for sensitive subgraph nodes. We initially disassembled the APK and extracted opcode counts from all smali files. Representing each opcode using 8 bits results in 256 possible opcodes. However, considering that several opcodes have similar functionalities and operate on the same data type, a final selection of 224 distinct opcodes is made. Table 3 illustrates 12 different opcode types.

Table 3. Twelve different types of opcodes.

id	1	2	3	4	5	6
opcode	nop	move	move/form16	move/16	move-wide	move-wide/from16
id	7	8	9	10	11	12
opcode	move-wide/16	move-object	move-object/from16	move-result	move-result-object	move-exception

This selection of opcodes not only reduces redundancy but also highlights the unique functions of applications. For example, different opcodes may perform similar operations on different data types or contexts within the application codebase. Streamlining to 224 dis-

tinct opcodes allows the representation to cover diverse functionalities while avoiding excessive complexity more effectively.

In summary, these opcodes are essential building blocks in the graph representation, encapsulating various operations that applications may perform. They play a vital role in comprehensively understanding application behaviors in the graph, enabling more fine-grained and concrete analysis that improves threat detection efficiency.

3.1.4. Third-Party Libraries (TPLs)

Apart from the internal APIs of the application itself, the API calls within the call graph also encompass APIs from separate functional modules. The application invokes these modules, which are developed by third-party library creators, to implement specific functionalities such as image processing, network communication, and database access, among others. Hence, this paper employs the LibRadar tool to identify third-party libraries (TPLs) within the application. This detection helps in recognizing APIs associated with independent functional modules. These third-party libraries might contain external code libraries utilized by the application to extend its functionality. By detecting and identifying these TPLs, this paper enhances the attributes of nodes in the call graph, enabling a more comprehensive understanding of the application's behavior.

3.2. Composition of Sensitive Subgraphs

Using the extracted call graph from Dex files for classification tasks can be time-consuming and costly during training. To address this, we propose a simplified call graph generation approach focusing on sensitive permission API calls and removing irrelevant nodes.

The APIs of sensitive permissions are set as the center nodes of the API call graph. First, we retain all neighbor nodes within a certain number of hops from the center nodes. Then, we perform semantic information identification and data flow analysis on all nodes within those hops, removing irrelevant and non-critical nodes to obtain a simplified API call graph. Taking a call graph with a sensitive node v_1 at the center and $\text{hop} = 4$ as an example, the specific rules for simplifying the graph are:

- Firstly, retain nodes directly connected to user-defined APIs;
- Secondly, retain nodes that can directly reach v_1 ;
- Thirdly, remove other isolated nodes.

Figure 3 illustrates the simplification process of a call graph. Here, v_{tpl} , v_{per} , v_{non} , and v_{usr} represent the node of APIs from independent functional modules, APIs related to permissions, and user-defined APIs, respectively.

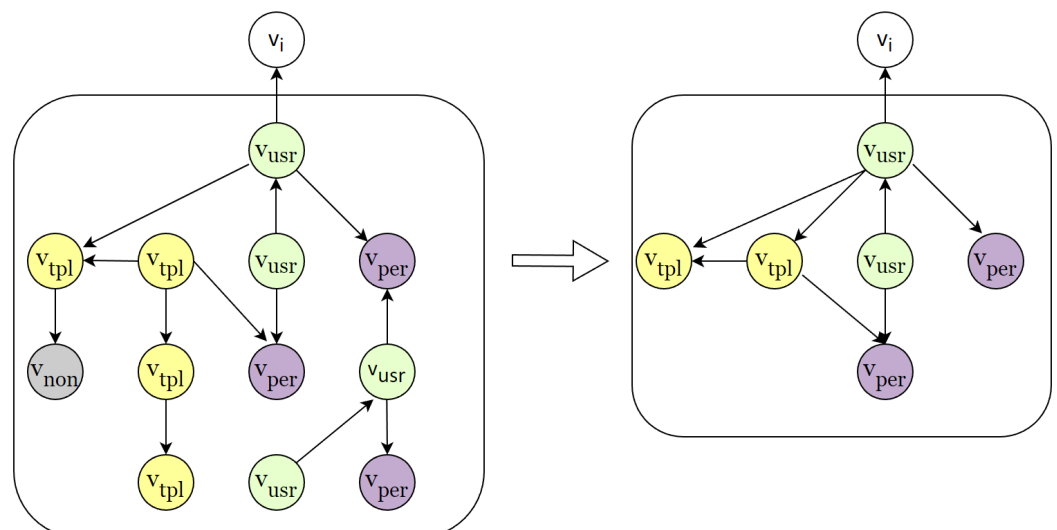


Figure 3. Sensitive subgraph after pruning.

3.3. Generating Behavior-Level Features

Each simplified call graph has a node matrix and a feature matrix. We retain the structural features of the simplified call graph. The node feature matrix of the simplified call graph is combined with the API features of sensitive permissions, operation codes, and third-party libraries to generate a new feature matrix, obtaining behavior-level features. Algorithm 1 shows the pseudocode for generating the behavior-level feature matrix.

Algorithm 1 Generating behavior feature matrix.

Require: $G, Ops, Perms, APIs$

Ensure: FV_final behavior feature matrix

```

1:  $m \leftarrow$  number of API nodes in  $G$ 
2:  $A \leftarrow \text{zeros}(m, m)$  ▷ Adjacency matrix
3: for  $i \leftarrow 1$  to  $m$  do
4:   for  $j \leftarrow 1$  to  $m$  do
5:     if  $G.\text{has\_edge}(i, j)$  then
6:        $A[i, j] \leftarrow 1$ 
7:  $FV\_opcode \leftarrow []$ 
8: for  $n$  in  $G.\text{nodes}$  do
9:    $Ops \leftarrow$  set of opcodes in  $n$ 
10:  for  $op$  in  $Ops$  do
11:     $FV\_opcode.\text{append}((op, \text{Freq}(op, n)))$ 
12:  $FV\_perms \leftarrow []$ 
13: for  $n$  in  $G.\text{nodes}$  do
14:  for  $p$  in  $Perms$  do
15:    if  $p$  in  $n$  then
16:       $FV\_perms.\text{append}(1)$ 
17:    else
18:       $FV\_perms.\text{append}(0)$ 
19:  $FV\_apis \leftarrow []$ 
20: for  $n$  in  $G.\text{nodes}$  do
21:  for  $api$  in  $APIs$  do
22:    if  $api$  in  $n$  then
23:       $FV\_apis.\text{append}(1)$ 
24:    else
25:       $FV\_apis.\text{append}(0)$ 
26:  $FV\_final \leftarrow FV\_opcode + FV\_perms + FV\_apis$ 
27: return  $FV\_final$ 

```

The main goal of this algorithm is to extract the behavior feature matrix FV_final from the simplified API call graph G .

To begin, we create an adjacency matrix that encapsulates the graphical structure of the simplified API call graph. In this scenario, we assume there are ' m ' API nodes remaining after the simplification process. The resulting adjacency matrix is of size ' $m \times m$ '. For any pair of API nodes ' i ' and ' j ', if there exists a calling relationship between them, the respective element in the matrix is set to '1'; otherwise, it is set to '0' (Lines 1–6).

Traversing all nodes in the call graph and counting the operation codes contained in each node and their frequencies, we store them in the operation code frequency feature vector FV_opcode (Lines 8–11). Similarly, we examine each node in the call graph to determine whether it contains any permissions from a given sensitive permission set, setting the corresponding position in the vector to 1 if yes and 0 otherwise, thus forming the permission feature vector FV_perms (Lines 12–18). The same procedure is followed to obtain the third-party API feature vector FV_apis (Lines 19–25).

Finally, the three feature vectors are concatenated to create the final behavior feature matrix FV_final (Line 26).

By extracting information from various aspects including graph structure, operation codes, permissions, and third-party API calls, the features can comprehensively represent a behavior. Together with the adjacency matrix, behavior-level features are generated, which can serve as input for subsequent model training.

3.4. Classification Model and Loss Function

In this section, we delve into the comprehensive design of our classification model, integrating Graph Neural Networks (GCNs), LSTM, and attention mechanisms to effectively process and analyze the combined features derived from distilled call graphs, encompassing permissions, operation codes, and additional behavioral traits. We aim to leverage these techniques to capture intricate dependencies within the call graph structures and empower the accurate detection of malicious software. Furthermore, we elucidate our approach in constructing the enhanced GCN model, its pivotal components, and the proposed loss function to refine the classification process for identifying various behaviors in the application nodes. Integrating these methodologies aims to improve the detection capabilities by considering structural and feature-based aspects of the graph data.

3.4.1. Our Classification Model

To effectively capture node information and exploit the intricate dependencies within graphs, we employ Graph Convolutional Networks (GCNs) [36], which excel at modeling graph-structured data. Furthermore, we integrate LSTM and attention mechanisms to model sequential dependencies and focus on crucial nodes. We aim to characterize application behavior and enable accurate malware detection by treating the Android malware detection task as a node classification problem on the sensitive subgraphs. We facilitate role modeling and classification processes for the application nodes by leveraging the enhanced GCN model. Figure 4 illustrates the improved GCN network model's overall structure, which outlines our approach's critical components.

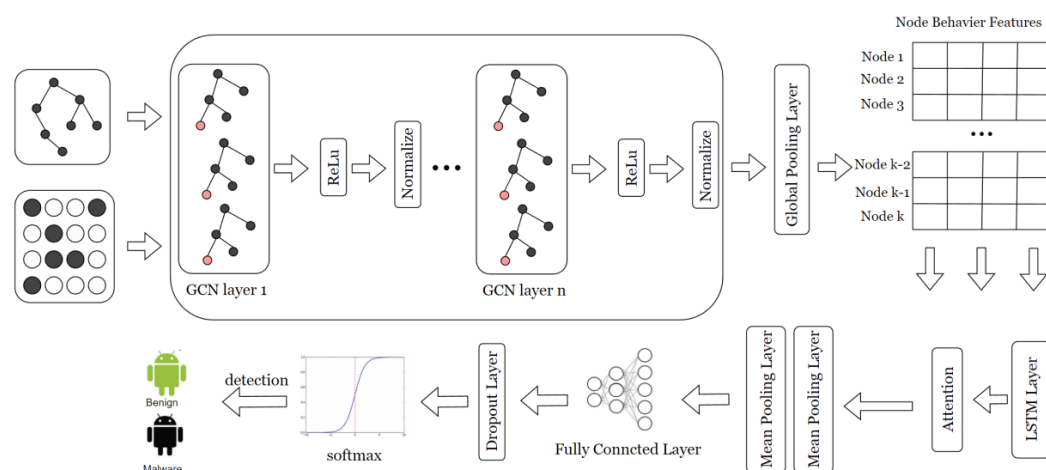


Figure 4. The structure of our classification model.

Use the API call graph adjacency matrix A and the behavioral feature matrix X constructed in the previous section as inputs to the GCN model. The adjacency matrix A reflects the calling relationships between APIs. In contrast, the feature matrix X incorporates behavior information of each API node, including operation codes, permissions, and third-party library invocations. The GCN model learns node embeddings on the graph structure through message passing and, under the guidance of the two matrices, obtains a graph representation that integrates topological structure and node features for effective classification and detection of sample behaviors.

Building upon the original GCN model, we use ChebConv convolution layers for efficient feature propagation while preserving the graph structure. To further process the pooled features, we incorporate LSTM and attention mechanisms, enabling the classification

of graph data. The overall architecture of the improved GCN network model is shown in Figure 4. It first goes through three layers of ChebConv convolutions, with activation functions and Dropout layers, i.e., MLP techniques, applied after each ChebConv layer for non-linear transformation and regularization. This is followed by global pooling, a combination of average and max pooling, and a superficial LSTM layer to obtain a weighted LSTM representation. The predicted category scores are accepted through the attention mechanism, fully connected layers, and a softmax layer after going through the final processing steps.

In our approach, we select the ChebConv [37] graph convolution model as a critical component for graph convolution modeling. Unlike traditional convolution kernels in the spectral domain, ChebConv leverages Chebyshev polynomials for recursive computation combined with linear transformations. This unique formulation enables effective feature propagation while retraining the inherent structure of the graph. The formula for ChebConv is provided below.

$$X^{l+1} = \sum_{k=0}^K \Theta^{(k)} \cdot T_k(\widetilde{L}) \cdot X^{(l)} \quad (1)$$

For our approach, the input feature matrix X^l denotes the node features at layer l , whereas the output feature matrix X^{l+1} represents the node features at the subsequent layer. The parameter K signifies the order of the Chebyshev polynomial utilized in the computation. To enable comprehensive analysis and classification of graph data, we incorporate LSTM and attention mechanisms.

This paper extracts the node features by combining global mean pooling and global maximum pooling techniques. Furthermore, the attention weights are normalized using the softmax function to ensure proper weighting. Through analysis, the time complexity of the GCN model proposed in this paper is $O(n)$, where n represents the number of nodes in the graph. This time complexity comes from the computational complexity of ChebConv graph convolution and the sequential computational complexity of LSTM. In the next section, we will define the loss function.

3.4.2. Loss Function

We use the loss function in MsDroid [12] as the loss function in this paper, it is shown in (2).

$$L = -((1 - y) \frac{1}{|G|} \sum_{g \in G} \log(1 - p_g) + y \min_{g \in G} \log(p_g)) \quad (2)$$

Specifically, the first part is the negative log-likelihood loss, which is represented as follows:

$$L = \sum (1 - y) \log(1 - p_g) \quad (3)$$

where $g \in G$, y is the actual label and is the predicted probability of the sample g by the model. The negative log-likelihood loss is commonly used in binary classification problems. When the actual label $y = 1$, the smaller the predicted probability of the sample g by the model, the larger the loss, and vice versa.

The second part is the minimum value loss, which is represented as follows:

$$L = y \min_{g \in G} \log(p_g) \quad (4)$$

where y is the actual label, and $\min$ represents selecting the minimum value of the probabilities in the set G . The objective of the minimum value loss is to minimize the difference between the model's predicted probability and the actual label by selecting the minimum probability from the set G . This helps train the model to make more accurate predictions and reduce the overall loss.

4. Experiment

This section provides a detailed overview of our experiments and performance evaluation methods. The objective is to delve into the intricacies of our experimental design and comparative performance assessments, gaining a profound understanding of the effectiveness and superiority of the model we propose in malicious software detection.

4.1. Dataset

We constructed the dataset using publicly available experimental datasets, which include the following components: benign applications from the open-access datasets CICMalDroid and Android Malware Dataset, as well as malicious applications from the Drebin dataset (DB) and Android Botnet dataset. These datasets provided a diverse range of samples for training and evaluating the performance of our model. The number of APK files included in the specific dataset is shown in Table 4.

Table 4. Evaluation dataset.

Dataset	Apps	Type
Drebin (DB)	4500	Malware
Android Botnet dataset	1000	Malware
CICMalDroid	4000	Benign
Benign_2015	500	Benign
Benign_2016	600	Benign
Benign_2017	68	Benign

We comprehensively compared our proposed and existing methods using various evaluation metrics, including accuracy, precision, recall, and F-score. The comparison results demonstrate the superiority of our approach in these tasks.

4.2. Evaluation Metrics

In this section, we randomly divided the dataset into a training set (80%) and a test set (20%). We performed several experimental comparisons to evaluate our approach's detection and runtime performance on the entire dataset.

Our evaluation methodology classifies malicious applications as positive samples and benign applications as negative samples. We utilize several commonly used evaluation metrics to assess the performance of our approach. These metrics include accuracy, precision, recall, and F1-score. The F1-score is a comprehensive metric calculated as the harmonic mean of accuracy and recall. The formulas for these evaluation metrics are as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP} \quad (5)$$

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (8)$$

4.3. Parameter Settings

In our study, we performed experiments on the parameters listed in Table 3 to train our model. We evaluated the performance based on precision, recall, accuracy, and F1-Score. The experimental results are shown in Table 5.

Table 5. Parameters of our proposed model.

Parameter	Value	Precision	Recall	Accuracy	F1-Score
Epoch	100	0.975	0.967	0.967	0.967
Epoch	200	0.982	0.977	0.977	0.987
Dropout	0.2	0.977	0.976	0.975	0.976
Dropout	0.25	0.982	0.975	0.977	0.978
GCN-layers	2	0.977	0.962	0.972	0.970
GCN-layers	3	0.980	0.970	0.975	0.978
Embedding-dimensions	32	0.965	0.984	0.968	0.974
Embedding-dimensions	64	0.977	0.988	0.978	0.982
Embedding-dimensions	128	0.982	0.975	0.977	0.978

Based on the experimental results, we adjusted the parameters of our model as follows: the embedding layer dimension was set to 128, the number of convolutional layers was set to 3, the dropout rate was set to 0.25, and the number of epochs was set to 200. Modifying these parameters aimed to enhance the model's performance based on the observed experimental outcomes.

4.4. Baseline Features and Models

4.4.1. Feature Selection Comparison

To validate the impact of the features used in this paper, we conducted a direct comparison by evaluating different features individually.

The experimental results, as presented in Table 6, demonstrate that our method achieves the best performance when utilizing the combined features of the call graph, permissions, and opcodes. This configuration successfully has 98.89% of malware instances.

Table 6. Performance of our work in feature selection.

Feature Selection	Precision	Recall	Accuracy	F1-Score
API-Call	0.6560	0.6779	0.7090	0.7771
API-Call + Opcode	0.7112	0.6920	0.6911	0.7800
API-Call + Permission	0.8000	0.7875	0.7970	0.8011
Our method	0.9889	0.9805	0.981	0.9847

4.4.2. Performance Comparison before and after Call Graph Simplification

In this section, we compared the time and memory expenses during the training and inference stages of the model before and after the simplification of the call graph. By documenting changes in time and memory, we aimed to comprehensively assess the impact of call graph simplification on algorithm performance. We randomly selected 1000 samples from our dataset, evenly split into 500 benign and 500 malicious applications for experimentation. The experimental results are presented in Table 7.

Table 7. Time and memory usage before and after API call graph simplification selection.

/	Training Time (Hours)	Inference Time (Hours)	Memory Consumption (GB)
Before Call Graph Simplification	1.1	0.5	10.20
After Call Graph Simplification	0.16	0.083	6.0

In the above table, we compare the model training and inference times and memory usage before and after applying call graph pruning. The results show that after call graph pruning is used, the model training and inference times are slightly reduced whereas the memory usage decreases. Through theoretical analysis, the time complexity of our model

is $O(n)$, where n represents the number of nodes in the call graph. Call graph pruning can reduce the number of nodes and edges in the graph, thus also reducing the time complexity.

4.4.3. Model Comparison

To assess the effectiveness of our proposed model, we initially transformed its core components into neural representations by independently stacking graphs. Subsequently, we incorporated two LSTM layers and attention mechanisms. The classifiers in our model and the compared networks were constructed by integrating a global average pooling layer, a fully connected layer, and a softmax function. We conducted multiple experiments using identical parameter settings to compare the performance of various graph convolutional network layers for graph structure classification.

The results of our experiments are displayed in Table 8. It can be observed that our proposed model outperforms the baseline GCN model. When compared to GraphConv and GATConv, our model achieves improvements of 2.96% and 2.5% in precision (Pre), 1.3% and 0.52% in accuracy (Acc), 0.34% in recall (Rec), and 1.12% and 0.23% in F1-score (F1), respectively. These results indicate that our proposed model surpasses the baseline model under the same experimental conditions.

Table 8. The performance of our work in model.

Model	Precision	Recall	Accuracy	F1-Score
GraphConv	0.9593	0.9771	0.9681	0.9735
GATConv	0.9646	0.9771	0.9759	0.9823
Our model	0.9889	0.9805	0.9811	0.9847

To demonstrate the experimental performance of ChebConv more concretely, we present in Figure 5 the experimental results for four evaluation metrics—Precision, Recall, Accuracy, and F1 score—as the number of epochs increases.

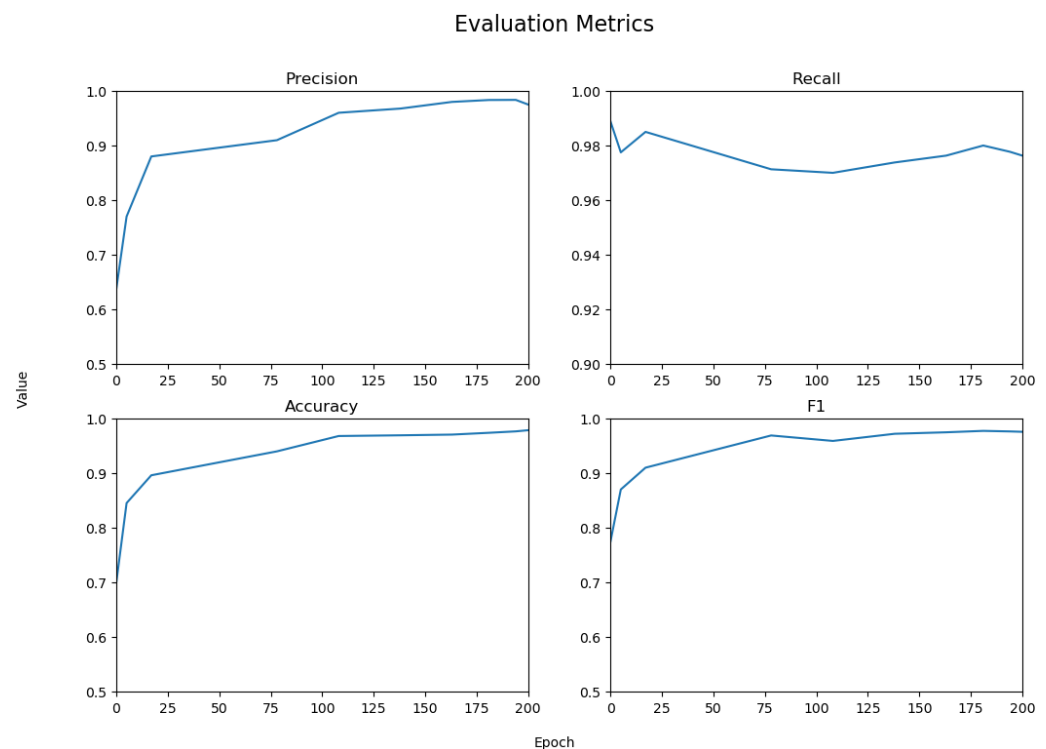


Figure 5. The four evaluation criteria of ChebConv.

As epochs increase, we observe significant improvement in precision, accuracy, and F1 score, but recall exhibits a relatively stable trend. This phenomenon may arise from several factors and does not necessarily imply a negative impact on our method's performance.

One possible reason for recall remaining stable is the balancing emphasis on boosting overall model performance while improving multiple indicators. The ChebConv layer might have been optimized to improve other metrics without significantly harming recall. Additionally, characteristics of the dataset or complexities captured by the ChebConv layer could lead to a stable recall metric, indicating consistent and reliable performance in identifying specific categories or patterns.

This observation highlights our model's capability to maintain a consistent recall rate while enhancing other key performance indicators. Our method effectively balances various aspects of performance, achieving stable and reliable classification capabilities for specific categories or patterns in the dataset.

In addition, we conducted ablation experiments to verify the effects of the LSTM and Attention modules, as shown in Table 9. When only using GCN, the model Precision is 0.962; after adding LSTM, it is improved to 0.971; when using both LSTM and Attention, the model Precision reaches 0.9889. The introduction of the LSTM layer can better model the order of API calls in the call graph, learning the sequential dependencies between nodes, which provides sequential dependency information that the GCN model fails to capture, thereby improving detection performance. The attention mechanism can adaptively adjust the weights of different nodes, enabling the model to focus more on critical suspicious API calls and reduce the influence of less relevant nodes. Incorporating the attention mechanism enhances the model's ability to automatically learn the essential nodes and structures in the call graph. The above ablation experiment results demonstrate that adding LSTM and Attention compensates for the deficiencies of the GCN model by integrating sequential information and modeling importance, significantly improving the final detection performance.

Table 9. Comparison with model.

Model	Precision	Recall	Accuracy	F1-Score
ChebConv	0.962	0.950	0.966	0.951
ChebConv + LSTM	0.971	0.974	0.966	0.960
ChebConv + LSTM- + Attention	0.9889	0.9805	0.9811	0.9847

4.5. Detection Performance Comparison with Existing Methods

To assess our proposed method's performance compared to other detection systems, we considered two additional detection methods from the literature.

- MsDroid [12] is an Android malware detection system that makes decisions by identifying malicious code snippets with interpretable explanations. Its key innovation is that it focuses on detecting snippets around sensitive APIs instead of the whole app. The method uses graph encoding to represent code snippets, combining code attributes and domain knowledge and then classifies them using a GNN.
- Drebin [38] employed an extensive static analysis approach to extract numerous features from manifest files and application source code, including permissions, API classes, and intents.
- MaMaDroid [39] focuses on extracting call graph files using the Android app taint analysis tool Flowdroid. The API sequences within these files are modeled as Markov chains, and feature vectors for the applications are constructed based on the probability of state transfer.
- MalDetGCN [40] proposes a novel method for Android malware detection using function call graphs and graph convolutional networks. This method first extracts function call graphs from the target programs and then transforms the function call graphs into feature vectors required as inputs for graph representation learning models.

Next, graph convolutional networks are utilized to perform deep learning on these feature vectors to obtain higher-level abstract representations, allowing more accurate malware detection.

As shown in Table 10, our proposed method achieves outstanding performance across all four evaluation metrics, especially in precision, where our approach reaches 98.89%, surpassing the other three detection methods. This significant improvement is that our process adopts a more elaborate and comprehensive feature extraction approach. First, we generate pruned call graph features based on API call graphs, then incorporate sensitive permissions, opcodes, and other information to create more behavioral-level graph features. This rich feature extraction enables our model better to capture the complex behavioral patterns of malicious code, thus improving detection accuracy. Finally, we employ an enhanced GCN model, integrating LSTM and attention mechanisms, which allows the model to handle long-range dependencies and global information better, further boosting malware detection performance. In summary, our method fully utilizes richer feature representations and a more robust model architecture, demonstrating superior performance in malware detection.

Table 10. Comparison with other related work.

Method	Precision	Recall	Accuracy	F1-Score	Runtime (Second)
MsDroid	0.970	0.972	0.970	0.971	182.0
Drebin	0.945	0.975	0.947	0.939	149.9
MaMaDroid	0.952	0.872	0.937	0.910	155.1
MalDetGCN	0.972	0.978	0.978	0.951	170.8
Our method	0.9889	0.9805	0.9811	0.9847	188.6
Std Dev	0.021	0.015	0.012	0.014	/

To evaluate the stability and variance of our method's results, we calculated and reported the standard deviations of the evaluation indicators of the proposed method, as shown in Table 10. As can be seen from the results, our proposed method exhibits lower standard deviations across all hands, with standard deviations of 0.021, 0.015, 0.012, and 0.014 for precision, recall, accuracy, and F1 score, respectively. This indicates that our malware detection method can obtain more stable and reproducible results, showing the high stability of the way.

In addition, we evaluated the computational cost of the classification process for Drebin, Mamadroid, and MalDetGCN, and our proposed method on 500 benign and 500 malicious apps randomly selected from the collection of more than ten thousand samples. As shown in Table 7, the proposed model incorporating GCNs, LSTM, and attention mechanism required more computational resources than other machine learning or GCN-only methods but achieved higher prediction accuracy. This was mainly because GCNs learn the feature representation in the graph structure, LSTM learns long-term dependencies in temporal sequences, and attention mechanisms adaptively focus on suspicious API calls. By combining these three components, the model can better mine patterns in the time graph, thereby improving final prediction performance. Compared to machine learning-based malware detection methods such as Drebin and Mamadroid, the prediction phase of our proposed model took longer due to the additional overhead of building and computing call graphs. However, through modeling software structure information using call graphs, our model achieves a higher F1 score (our model = 98.22%, Drebin = 93.9%, Mamadroid = 91.0%). To reduce unnecessary computation, we used model optimization methods such as using only three layers in GCNs, LSTM hidden layer size of 1, and optimized code such as parallel computation on GPUs and reducing intermediate data transfer. When testing on 1000 application samples, the prediction time of our model was 188.6 s, whereas Drebin and Mamadroid were 149.9 s and 155.1 s, respectively. Despite taking slightly longer than MalDetGCN (a GCN-based graph neural network model), incorporating LSTM and attention mechanism allows for better modeling of the sequential

dependencies and importance of suspicious APIs within the call graph, resulting in improved detection performance. Although the prediction time is longer than single-GCN models, the main computational overhead comes from modeling called graph structure information compared to non-graph-based methods like Drebin and Mamadroid. Given the improvement in detection performance, the increase in prediction time is acceptable. Future work will further balance efficiency and performance through model compression and quantization.

5. Discussion

Our malware detection method prompts deep reflection on ethical issues in technology applications, and we recognize several vital points:

- Privacy protection and sensitive data handling: Analysis of sensitive user data (such as permissions and API calls) may pose potential threats to user privacy. To prevent privacy breaches, we strive to introduce robust data anonymization mechanisms and adopt strict data protection measures to ensure the secure processing of user information.
- Risk of technology abuse: We recognize that malicious actors could misuse our technical capabilities for surveillance purposes, and lack of user consent may raise ethical concerns. Therefore, we seek technical solutions, such as introducing user notification and permission processes, to ensure users' control over their data and effectively mitigate potential abuse risks.
- Model prediction biases: We also acknowledge that model predictions may harbor certain tendencies leading to unfair outcomes for specific groups. We will actively conduct bias assessments and take measures to address this issue to ensure that our technology applications treat all users fairly.

By considering these ethical dimensions holistically, we are committed to technological innovation and ensuring respect and protection of users' privacy rights while developing new technologies. We will continue to strive tirelessly to develop more responsible and sustainable malware detection technologies, contributing to enhanced mobile security.

6. Conclusions and Future Work

In this paper, we propose a novel approach for malware detection based on Graph Convolutional Networks (GCNs). Our approach tackles the task as an end-to-end node classification problem, eliminating the need for complex graph-matching techniques. Our approach begins by extracting an API call graph from the APK. Subsequently, we propose a method for simplifying the call graph using sensitive permission APIs. This simplification reduces the processing time for features and lowers the complexity of the API call graph. We enhance the model by incorporating multiple features such as sensitive permissions, opcodes, and others to enrich the semantics of the API. Furthermore, we improve the GCN model by adding LSTM layers and an attention mechanism to boost the model's performance. Our experimental results demonstrate that our method outperforms baseline methods.

The advantages of our approach include call graph simplification, reduced complexity, and the avoidance of dynamic behavior feature extraction. By integrating multiple features, we enrich the semantics of the API, which is crucial for improving detection accuracy. Moreover, we found that using GCNs along with LSTM and the attention mechanism can directly process graph results without further processing while capturing subtle relationships between the network flows, which is vital for deep learning classification.

Although our method makes significant progress in malicious software detection, there are several limitations worth noting:

- Our approach heavily depends on static features and cannot fully capture dynamic behaviors, potentially leading to some malicious activities being missed.
- Although we simplified the call graph, there is still a risk of information loss in specific scenarios.

- Our model may need more complexity and computational resource challenges when handling large-scale datasets.

Future work should continue to refine our approach by introducing dynamic features to capture real-time changes in malicious behavior, improving model robustness, and optimizing time complexity and resource usage further to enhance the comprehensiveness and reliability of our technique.

Moreover, our research brings key advancements in malicious software detection. Integrating our method into existing security software can lead to increased security through the precise detection of malicious behavior. However, integration could face challenges such as dataset adaptability and real-time requirements. Future work must focus on overcoming these challenges to ensure a greater impact on practical security applications.

Author Contributions: Methodology, Q.X.; software, Q.X.; validation, S.Y.; formal analysis, D.Z. and X.L.; investigation, Q.X. and L.X.; writing—original draft, Q.X.; writing—review and editing, S.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the National Natural Science Foundation of China (62172244), the Shandong Provincial Natural Science Foundation (ZR2020YQ06), the Taishan Scholars Program (tsqn202211210), the Young Innovation team of colleges and universities in Shandong Province (2021KJ001), the Pilot Project for Integrated Innovation of Science, Education, and Industry of Qilu University of Technology (Shandong Academy of Sciences) (2022JBZ01-01), the Graduate Education and Teaching Reform Research Project of Shandong Province (SDYJG21177), the Education Reform Project of Qilu University of Technology (2021yb63), and The Innovation Ability Promotion Project for Small and Medium-sized Technology-based Enterprise of Shandong Province (2023TSGC0150).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Qiu, J.; Zhang, J.; Luo, W.; Pan, L.; Nepal, S.; Xiang, Y. A survey of android malware detection with deep neural models. *ACM Comput. Surv. (CSUR)* **2020**, *53*, 126. [\[CrossRef\]](#)
2. Fan, M.; Luo, X.; Liu, J.; Wang, M.; Nong, C.; Zheng, Q.; Liu, T. Graph embedding based familial analysis of android malware using unsupervised learning. In Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 25–31 May 2019; pp. 771–782.
3. Zhang, X.; Zhang, Y.; Zhong, M.; Ding, D.; Cao, Y.; Zhang, Y.; Zhang, M.; Yang, M. Enhancing state-of-the-art classifiers with api semantics to detect evolved android malware. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, 9–13 November 2020; pp. 757–770.
4. Mathur, A.; Podila, L.M.; Kulkarni, K.; Niyaz, Q.; Javaid, A.Y. NATICUSdroid: A malware detection framework for Android using native and custom permissions. *J. Inf. Secur. Appl.* **2021**, *58*, 102696. [\[CrossRef\]](#)
5. Narayanan, A.; Yang, L.; Chen, L.; Jinliang, L. Adaptive and scalable android malware detection through online learning. In Proceedings of the 2016 International Joint Conference on Neural Networks (IJCNN), Vancouver, BC, Canada, 24–29 July 2016; pp. 2484–2491.
6. Zhang, N.; Tan, Y.A.; Yang, C.; Li, Y. Deep learning feature exploration for android malware detection. *Appl. Soft Comput.* **2021**, *102*, 107069. [\[CrossRef\]](#)
7. Vinod, P.; Zemmari, A.; Conti, M. A machine learning based approach to detect malicious android apps using discriminant system calls. *Future Gener. Comput. Syst.* **2019**, *94*, 333–350.
8. Gao, H.; Guo, C.; Wu, Y.; Dong, N.; Hou, X.; Xu, S.; Xu, J. AutoPer: Automatic recommender for runtime-permission in android applications. In Proceedings of the 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC), Milwaukee, WI, USA, 15–19 July 2019; Volume 1, pp. 107–116.
9. Arora, A.; Peddoju, S.K.; Conti, M. Permpair: Android malware detection using permission pairs. *IEEE Trans. Inf. Forensics Secur.* **2019**, *15*, 1968–1982. [\[CrossRef\]](#)
10. Zhang, Y.; Sui, Y.; Pan, S.; Zheng, Z.; Ning, B.; Tsang, I.; Zhou, W. Familial clustering for weakly-labeled android malware using hybrid representation learning. *IEEE Trans. Inf. Forensics Secur.* **2019**, *15*, 3401–3414. [\[CrossRef\]](#)
11. Mirzaei, O.; Suarez-Tangil, G.; de Fuentes, J.M.; Tapiador, J.; Stringhini, G. Andrensemble: Leveraging api ensembles to characterize android malware families. In Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security, Auckland, New Zealand, 9–12 July 2019; pp. 307–314.

12. He, Y.; Liu, Y.; Wu, L.; Yang, Z.; Ren, K.; Qin, Z. MsDroid: Identifying Malicious Snippets for Android Malware Detection. *IEEE Trans. Dependable Secur. Comput.* **2023**, *20*, 2025–2039. [\[CrossRef\]](#)
13. Desnos, A.; Gueguen, G. Androguard Documentation. Obtenido de Androguard. 2018. Available online: <https://andro-guard.readthedocs.io/en/latest> (accessed on 23 October 2023).
14. Liu, K.; Xu, S.; Xu, G.; Zhang, M.; Sun, D.; Liu, H. A review of android malware detection approaches based on machine learning. *IEEE Access* **2020**, *8*, 124579–124607. [\[CrossRef\]](#)
15. Mahindru, A.; Sangal, A. MLDroid—Framework for Android malware detection using machine learning techniques. *Neural Comput. Appl.* **2021**, *33*, 5183–5240. [\[CrossRef\]](#)
16. Gibert, D.; Mateu, C.; Planes, J.; Vicens, R. Using convolutional neural networks for classification of malware represented as images. *J. Comput. Virol. Hacking Tech.* **2019**, *15*, 15–28. [\[CrossRef\]](#)
17. Vasan, D.; Alazab, M.; Wassan, S.; Safaei, B.; Zheng, Q. Image-Based malware classification using ensemble of CNN architectures (IMCEC). *Comput. Secur.* **2020**, *92*, 101748. [\[CrossRef\]](#)
18. Aafer, Y.; Du, W.; Yin, H. Droidapiminer: Mining api-level features for robust malware detection in android. In *SecureComm 2013: Security and Privacy in Communication Networks, Proceedings of the 9th International ICST Conference, Sydney, NSW, Australia, 25–28 September 2013; Revised Selected Papers 9*; Springer: Berlin/Heidelberg, Germany, 2013; pp. 86–103. [\[CrossRef\]](#)
19. Deshotels, L.; Notani, V.; Lakhota, A. Droidlegacy: Automated familial classification of android malware. In Proceedings of the ACM SIGPLAN on Program Protection and Reverse Engineering Workshop 2014, San Diego, CA, USA, 25 January 2014; pp. 1–12. [\[CrossRef\]](#)
20. Kim, T.; Kang, B.; Rho, M.; Sezer, S.; Im, E.G. A multimodal deep learning method for android malware detection using various features. *IEEE Trans. Inf. Forensics Secur.* **2018**, *14*, 773–788. [\[CrossRef\]](#)
21. Feizollah, A.; Anuar, N.B.; Salleh, R.; Suarez-Tangil, G.; Furnell, S. Androdialysis: Analysis of android intent effectiveness in malware detection. *Comput. Secur.* **2017**, *65*, 121–134. [\[CrossRef\]](#)
22. Seraj, S.; Khodambashi, S.; Pavlidis, M.; Polatidis, N. HamDroid: Permission-based harmful android anti-malware detection using neural networks. *Neural Comput. Appl.* **2022**, *34*, 15165–15174. [\[CrossRef\]](#)
23. Fan, M.; Liu, J.; Wang, W.; Li, H.; Tian, Z.; Liu, T. Dapasa: Detecting android piggybacked apps through sensitive subgraph analysis. *IEEE Trans. Inf. Forensics Secur.* **2017**, *12*, 1772–1785. [\[CrossRef\]](#)
24. Fan, M.; Liu, J.; Luo, X.; Chen, K.; Tian, Z.; Zheng, Q.; Liu, T. Android malware familial classification and representative sample selection via frequent subgraph analysis. *IEEE Trans. Inf. Forensics Secur.* **2018**, *13*, 1890–1905. [\[CrossRef\]](#)
25. Narayanan, A.; Meng, G.; Yang, L.; Liu, J.; Chen, L. Contextual weisfeiler-lehman graph kernel for malware detection. In Proceedings of the 2016 International Joint Conference on Neural Networks (IJCNN), Vancouver, BC, Canada, 24–29 July 2016; pp. 4701–4708.
26. Xu, X.; Liu, C.; Feng, Q.; Yin, H.; Song, L.; Song, D. Neural network-based graph embedding for cross-platform binary code similarity detection. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Dallas, TX, USA, 30 October–3 November 2017; pp. 363–376.
27. Hassen, M.; Chan, P.K. Scalable function call graph-based malware classification. In Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy, Scottsdale, AZ, USA, 22–24 March 2017; pp. 239–248.
28. Chen, T.; Mao, Q.; Yang, Y.; Lv, M.; Zhu, J. Tinydroid: A lightweight and efficient model for android malware detection and classification. *Mob. Inf. Syst.* **2018**, *2018*, 4157156. [\[CrossRef\]](#)
29. Alzaylaee, M.K.; Yerima, S.Y.; Sezer, S. DL-Droid: Deep learning based android malware detection using real devices. *Comput. Secur.* **2020**, *89*, 101663. [\[CrossRef\]](#)
30. Hemalatha, J.; Roseline, S.A.; Geetha, S.; Kadry, S.; Damaševičius, R. An efficient densenet-based deep learning model for malware detection. *Entropy* **2021**, *23*, 344. [\[CrossRef\]](#)
31. Azeez, N.A.; Odufuwa, O.E.; Misra, S.; Oluranti, J.; Damaševičius, R. Windows PE malware detection using ensemble learning. *Informatics* **2021**, *8*, 10. [\[CrossRef\]](#)
32. Damaševičius, R.; Venčkauskas, A.; Toldinas, J.; Grigaliūnas, Š. Ensemble-based classification using neural networks and machine learning models for windows pe malware detection. *Electronics* **2021**, *10*, 485. [\[CrossRef\]](#)
33. Zhang, D.; Kabuka, M.R. Protein family classification with multi-layer graph convolutional networks. In Proceedings of the 2018 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Madrid, Spain, 3–6 December 2018; pp. 2390–2393.
34. Gao, H.; Cheng, S.; Zhang, W. GDroid: Android malware detection and classification with graph convolutional network. *Comput. Secur.* **2021**, *106*, 102264. [\[CrossRef\]](#)
35. Yan, J.; Yan, G.; Jin, D. Classifying malware represented as control flow graphs using deep graph convolutional neural network. In Proceedings of the 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Portland, OR, USA, 24–27 June 2019; pp. 52–63.
36. Kipf, T.N.; Welling, M. Semi-supervised classification with graph convolutional networks. *arXiv* **2016**, arXiv:1609.02907.
37. Defferrard, M.; Bresson, X.; Vandergheynst, P. Convolutional neural networks on graphs with fast localized spectral filtering. In Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS’16), Barcelona, Spain, 5–10 December 2016; Volume 29.
38. Arp, D.; Spreitzerbarth, M.; Hubner, M.; Gascon, H.; Rieck, K.; Siemens, C. Drebin: Effective and explainable detection of android malware in your pocket. In Proceedings of the NDSS, San Diego, CA, USA, 23–26 February 2014; Volume 14, pp. 23–26.

39. Mariconti, E.; Onwuzurike, L.; Andriotis, P.; De Cristofaro, E.; Ross, G.; Stringhini, G. Mamadroid: Detecting android malware by building markov chains of behavioral models. *arXiv* **2016**, arXiv:1612.04433.
40. Vinayaka, K.; Jaidhar, C. Android malware detection using function call graph with graph convolutional networks. In Proceedings of the 2021 2nd International Conference on Secure Cyber Computing and Communications (ICSCCC), Jalandhar, India, 26–28 May 2021; pp. 279–287.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.